

Project report on
Haptic Feedback using Robotic Arm and Raspberry
Pi

Submitted by

- Aryan Nayak – 2341014089
- Ansuman Sahu – 2341019234
- Dhruv Mohapatra – 2341013364
- Akash Kumar Panda – 2341016041
- Pratyush Kumar Mohanty – 2341013241
- Priyanshu Priyadarshan Sahu 2341019221

B. Tech. (CSE (IOT) 5th Semester (Section–23412B1)



CENTRE FOR INTERNET OF THINGS
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
Institute of Technical Education and Research

SIKSHA 'O' ANUSANDHAN
DEEMED TO BE UNIVERSITY

Bhubaneswar, Odisha, India.
(January, 2026)

Abstract

Haptic feedback systems enhance human–robot interaction by allowing users not only to control robotic devices but also to experience a sense of touch or force response. This project presents the design and implementation of a Haptic feedback–based robotic arm system using a Raspberry Pi as the main processing unit and a 4-DOF acrylic robotic arm as the actuation mechanism. A wearable glove equipped with flex sensors is used to capture finger movements, which are transmitted wirelessly using an ESP32 microcontroller. The Raspberry Pi processes the received gesture data and controls multiple servo motors through a PCA9685 servo driver to replicate the hand movements on the robotic arm. To implement haptic feedback, a load cell with HX711 amplifier detects applied force at the robotic arm gripper, and a vibration motor provides tactile feedback to the user. The proposed system demonstrates real-time gesture control with force feedback, offering a cost-effective and intuitive solution for applications in robotics education, teleoperation, prosthetics, and assistive technologies.

Contents

<u>Abstract</u>	2
<u>Contents</u>	3
1. Introduction	
2. Background and Motivation	
3. Aim and Objectives	
4. Scope of the Project	
5. Project Management	
6. Literature Survey	
7. Theoretical Background	
8. System Architecture and Design	
9. Hardware Requirements	
10. Software Requirements	
11. Working Principle	
12. Project Development Methodology	
13. Testing and Performance Analysis	

14. Results and Discussion

15. Conclusion

16. Limitations

17. Future Scope

18. References

19. Appendix

References.....31

Appendix

01.....32

A01.1. Code Listing43

A01.2. Main Code43

A01.3. Libraries43

Appendix 02.....43

A02.1. Project Proposal Form43

A02.2. Project Management.....43

A02.3. Bill of Material.....44

Appendix

03.....49

A03.1. Data Sheets45

Chapter 01: Introduction

1.1. Introduction

Robotics has become an essential part of modern technology, playing a significant role in industries such as manufacturing, healthcare, automation, and assistive systems. One of the major challenges in robotics is achieving natural and intuitive interaction between humans and machines. Traditional robotic control methods rely on switches, joysticks, or programmed commands, which are not always user-friendly or responsive.

Haptic feedback technology addresses this limitation by enabling users to feel physical sensations such as force, pressure, or vibration while interacting with robotic systems. By combining gesture control with haptic feedback, robots can provide a more realistic and immersive user experience.

This project focuses on developing a haptic feedback-enabled robotic arm system using Raspberry Pi and a 4-DOF acrylic robotic arm. The system allows a user to control the robotic arm using hand gestures while receiving tactile feedback based on the interaction of the robotic arm with objects.

1.2. Background

Early robotic systems were controlled using rigid wired interfaces with limited feedback to the user. Although these systems were functional, they lacked adaptability and intuitive control. With advancements in embedded systems, sensors, and wireless communication, it has become possible to design wearable and gesture-based robotic control systems.

Gesture-controlled robotic arms provide a natural way of controlling robots, especially in environments where direct physical interaction is not possible. However, most gesture-controlled systems do not provide feedback to the user, making it difficult to judge grip force or object interaction. This limitation can lead to inaccurate manipulation or damage to objects.

The integration of haptic feedback, such as force sensing and vibration response, solves this problem by informing the user about the amount of force applied by the robotic arm. This project builds upon these concepts to create a complete closed-loop human–robot interaction system.

1.3. Project Objectives

The main objectives of this project are:

- To design a gesture-controlled robotic arm using a 4-DOF acrylic structure
- To use Raspberry Pi as the main processing and control unit
- To capture hand gestures using flex sensors mounted on a glove
- To transmit gesture data wirelessly using an ESP32 microcontroller
- To control servo motors using a PCA9685 PWM servo driver
- To implement haptic feedback using a load cell and vibration motor
- To develop a low-cost, portable, and scalable system

1.4. Scope

The scope of this project includes:

- Inclusions
- Gesture-based control of a 4-DOF robotic arm
- Wireless communication between glove and control unit
- Real-time servo motor actuation
- Force-based haptic feedback using load cell and vibration motor
- Educational and demonstration-level implementation
- Exclusions
- Vision-based gesture recognition

1.5. Project Management

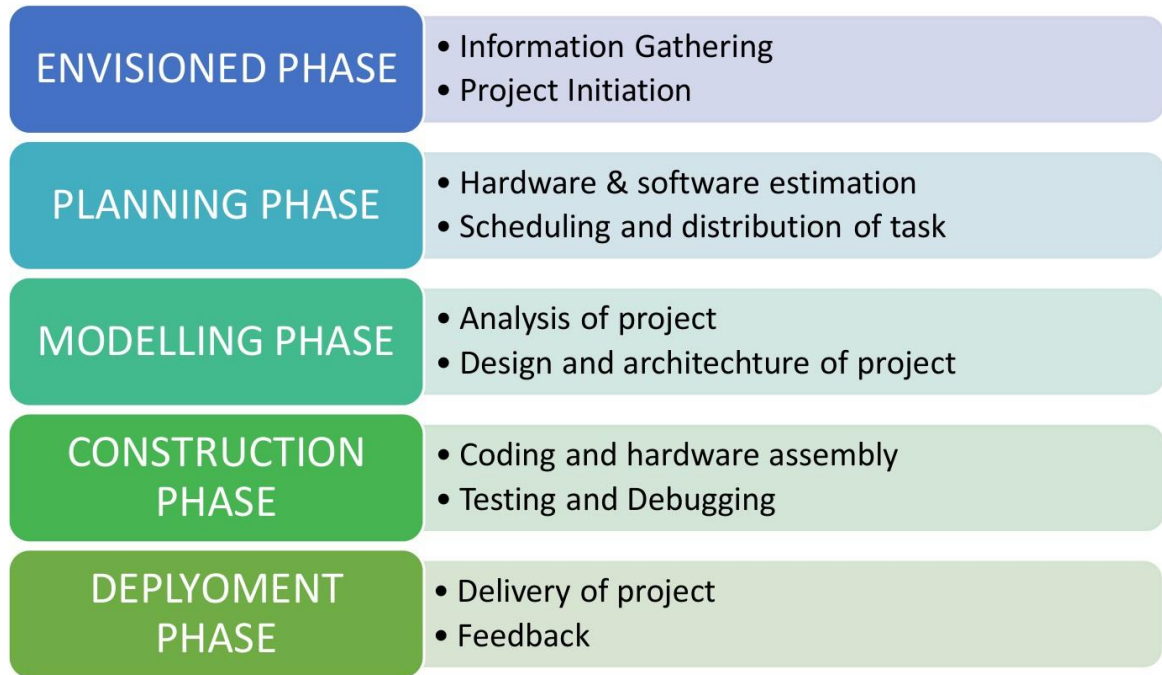


Figure 1. Model of phases in project management.

The project was executed using a systematic development approach consisting of the following phases:

1. Requirement Analysis
Selection of hardware components and identification of system requirements.
2. System Design
Designing the gesture input unit, processing unit, robotic arm, and haptic feedback mechanism.
3. Hardware Implementation
Assembly of the 4-DOF acrylic robotic arm, integration of servo motors, sensors, and power modules.
4. Software Development
Programming the ESP32 and Raspberry Pi for data transmission, processing, and motor control.
5. System Integration and Testing
Integrating hardware and software modules and validating real-time performance.
6. Documentation and Reporting
Preparing detailed project documentation and final report.

1.6. Overview and Benefits

The proposed system consists of a gesture input unit, processing unit, robotic arm, and haptic feedback unit. A glove embedded with flex sensors captures finger movements and sends the data to an ESP32 microcontroller. The ESP32 transmits the gesture data wirelessly to the Raspberry Pi using Bluetooth or Wi-Fi communication.

The Raspberry Pi processes the received data and generates PWM signals through the PCA9685 servo driver to control the servo motors mounted on the 4-DOF acrylic robotic arm. When the robotic arm interacts with an object, a load cell sensor placed at the gripper measures the applied force. This force information is sent back to the glove, where a vibration motor provides tactile feedback to the user, completing the haptic feedback loop.

1.6.1. Benefits of the Proposed System

Intuitive Human–Robot Interaction

The proposed system enables intuitive human–robot interaction by allowing the user to control the robotic arm through natural hand gestures rather than conventional input devices such as joysticks, switches, or keyboards. The use of flex sensors mounted on a wearable glove captures finger movements directly, making the interaction simple and instinctive. This natural mode of control significantly reduces the learning curve for users and improves operational efficiency, especially in applications requiring precise manipulation. By mimicking human hand movements, the robotic arm responds in a manner that feels natural and closely aligned with human intent..

Wearable and User-Friendly Design

The gesture control unit is implemented in the form of a lightweight and wearable glove, ensuring user comfort during prolonged operation. The sensors and electronic components are arranged in a compact manner, allowing free and unrestricted hand movement. This user-friendly design makes the system suitable for continuous use in educational demonstrations, training environments, and experimental applications. The wearable nature

of the glove enhances portability and ensures that the system can be easily adapted for different users without complex setup procedures.

Real-Time Response

Real-time responsiveness is a critical requirement for effective gesture-controlled robotic systems. In the proposed project, hand movements are captured by the sensors and transmitted wirelessly to the Raspberry Pi with minimal latency using the ESP32 communication module. The Raspberry Pi processes the incoming data and generates control signals for the servo motors almost instantaneously. As a result, the robotic arm mirrors the user's hand movements in real time, providing smooth and accurate motion. This low-latency operation is particularly important for tasks involving delicate object handling and precise positioning.

Reliable and Accurate Performance

The system is based on sensor-driven gesture detection, which ensures reliable performance under various environmental conditions. Unlike vision-based systems, the proposed approach is not affected by changes in lighting, background clutter, or camera alignment. The use of flex sensors and force sensing through the load cell provides consistent and accurate input data. This reliability makes the system suitable for indoor and controlled environments where dependable robotic operation is essential. Additionally, the integration of haptic feedback further improves accuracy by allowing the user to sense the applied force during object manipulation.

Scalable and Flexible Design

The proposed system is designed with scalability and flexibility in mind. Additional sensors, such as more flex sensors or advanced force sensors, can be easily integrated to enhance functionality. The control algorithms running on the Raspberry Pi can be modified or upgraded to support additional degrees of freedom, advanced motion control, or intelligent feedback mechanisms. Furthermore, the system can be extended to support IoT-based remote operation or AI-assisted control in future implementations. This flexibility ensures that the project can evolve beyond its current scope and be adapted for advanced research and real-world applications.

Organization of the Report

This project report is organized into seven chapters, each describing a specific phase of the design, development, and implementation of the **Haptic Feedback robotic arm system**.

Chapter 1: Introduction

This chapter introduces the concept of haptic feedback and gesture-based robotic control and highlights the importance of intuitive human–robot interaction. It presents the background of the project, objectives, scope, benefits of the proposed system, and an overview of project planning and management.

Chapter 2: Literature Survey

This chapter reviews existing research and previous work related to gesture-controlled robotic arms, haptic feedback systems, wearable sensor-based interfaces, force sensing mechanisms, and wireless communication techniques. It identifies the limitations of existing systems and establishes the motivation for developing the proposed haptic feedback robotic arm.

Chapter 3: System Architecture and Design

This chapter describes the overall architecture of the proposed system. It explains the design of the gesture input unit, processing unit, robotic arm, and haptic feedback module. The chapter includes the system block diagram, data flow, communication between ESP32 and Raspberry Pi, and the control strategy used for the 4-DOF acrylic robotic arm.

Chapter 4: Hardware Implementation

This chapter presents a detailed description of the hardware components and interfacing techniques used in the implementation of the haptic feedback–enabled robotic arm system. The major hardware components include the Raspberry Pi 4, ESP32 microcontroller, flex sensors for finger movement detection, load cell with HX711 amplifier for force sensing, coin-type vibration motor for haptic feedback, PCA9685 PWM servo driver, SG90 servo motors, and the required power supply modules.

In this project, the Arduino IDE is used to program the ESP32 microcontroller for reading finger movement data from the flex sensors and

transmitting the processed data wirelessly. The ESP32 continuously captures analog sensor values corresponding to finger bending and sends this data to the Raspberry Pi.

The Raspberry Pi 4 acts as the central processing and control unit of the system. Communication between the ESP32 and Raspberry Pi is established using wireless connectivity, and the Raspberry Pi is accessed and monitored using RealVNC, which enables remote desktop control and system monitoring during development and testing. RealVNC provides a convenient interface to run programs, visualize data, and debug the system without requiring direct physical access to the Raspberry Pi.

This chapter also explains the circuit connections, including sensor interfacing, power distribution, and servo motor connections through the PCA9685 driver. Additionally, it describes the physical assembly of the 4-DOF acrylic robotic arm, mounting of servo motors, placement of the load cell at the gripper, and integration of the wearable glove unit used for gesture input and haptic feedback.

Chapter 5: Software Implementation

This chapter details the software development process used in the project. It includes reading finger movement data using the Arduino IDE on ESP32, wireless data transmission between ESP32 and Raspberry Pi, Python-based processing on Raspberry Pi, servo motor control using PWM signals, and haptic feedback generation logic. The use of RealVNC for monitoring and controlling the Raspberry Pi is also discussed.

Chapter 6: Results and Performance Analysis

This chapter presents and analyzes the experimental results obtained from the implemented system. It discusses real-time system responsiveness, accuracy of gesture-based robotic arm control, effectiveness of haptic feedback, servo motor performance, and overall system reliability under different operating conditions.

Chapter 7: Conclusion and Future Scope

This chapter summarizes the outcomes of the project and evaluates the effectiveness of the proposed haptic feedback robotic arm system. It also suggests future enhancements such as improving force feedback accuracy, adding more degrees of freedom, integrating advanced control or AI-based algorithms, and enabling IoT-based remote operation.

Chapter 02: Literature Survey

2.1 Related Works

Gesture-controlled robotic systems and haptic feedback technologies have gained significant attention in recent years due to their applications in teleoperation, medical robotics, prosthetics, rehabilitation, and industrial automation. Researchers have explored various approaches to enable intuitive human-robot interaction, ranging from vision-based gesture recognition to sensor-based wearable systems. Each approach offers distinct advantages but also presents certain limitations. This section reviews important research works related to gesture-controlled robotic arms and haptic feedback systems, with particular emphasis on smart glove-based and sensor-driven solutions relevant to the proposed project.

2.1.1 Haptic Feedback in Robotic Manipulation Systems

Early haptic robotic systems were primarily developed for **teleoperation and surgical robotics**, where precise force control is critical. These systems employed force sensors at the end effector to measure interaction forces and relay this information back to the operator through mechanical or electrical feedback devices. Studies demonstrated that force feedback significantly improves manipulation accuracy and reduces the risk of excessive force application.

However, many early haptic systems were complex, expensive, and required specialized hardware, limiting their use to industrial or medical environments. These limitations motivated the development of simpler and more affordable haptic feedback solutions for educational and research applications.

2.1.2 Force Sensing Techniques in Haptic Systems

Force sensing is a key component of haptic feedback systems. Researchers have explored various force sensing methods, including force-sensitive resistors (FSRs), strain gauges, and load cells. Among these, load cells are widely used due to their accuracy and reliability in measuring applied force.

Load cells typically produce very small electrical signals, which require amplification and analog-to-digital conversion. Modules such as the HX711 amplifier are commonly used to interface load cells with embedded controllers. Several studies have successfully implemented load cell-based force sensing in robotic grippers to detect grip force and provide feedback to users.

The proposed project adopts this approach by using a 5 kg load cell with an HX711 amplifier to measure gripping force at the robotic arm's gripper.

2.1.3 Tactile Feedback Mechanisms

Tactile feedback mechanisms are used to convey force or contact information back to the user. Common methods include vibration motors, linear resonant actuators (LRAs), and haptic actuators. Among these, vibration motors are widely used in low-cost haptic systems due to their simplicity, compact size, and low power consumption.

Several research works have shown that vibration-based feedback is effective in conveying information about contact, pressure, and force magnitude. By varying vibration intensity or duration, users can perceive different levels of force applied by a robotic system.

In this project, a coin-type vibration motor is used to provide tactile feedback to the user based on force data measured by the load cell.

2.1.4 Embedded System-Based Haptic Control Architectures

Despite advancements, existing haptic robotic systems still face several challenges. High-end haptic systems are often expensive and complex, making them unsuitable for educational or low-cost applications. Some systems lack portability, while others provide limited feedback resolution.

Additionally, many proposed systems focus on either haptic feedback or robotic control independently, rather than integrating both into a unified and user-friendly solution. Power management, scalability, and ease of implementation also remain key challenges.

These limitations highlight the need for a cost-effective, portable, and integrated haptic feedback robotic arm system.

2.1.5 Limitations of Existing Systems

Despite advancements, existing haptic robotic systems still face several challenges. High-end haptic systems are often expensive and complex, making them unsuitable for educational or low-cost applications. Some systems lack portability, while others provide limited feedback resolution.

Additionally, many proposed systems focus on either haptic feedback or robotic control independently, rather than integrating both into a unified and user-friendly solution. Power management, scalability, and ease of implementation also remain key challenges.

These limitations highlight the need for a cost-effective, portable, and integrated haptic feedback robotic arm system

2.1.6 Relevance to the Proposed System

The proposed haptic feedback robotic arm using Raspberry Pi and a 4-DOF acrylic robotic arm builds upon existing research while addressing identified limitations. By integrating force sensing using a load cell, tactile feedback using a vibration motor, sensor-based gesture control, ESP32-based wireless communication, and Raspberry Pi-based processing, the system provides real-time force-aware robotic manipulation in an affordable and scalable design.

Chapter 03: Theoretical Aspects

3.1 Robotics Programming and Workshop

Robotics Programming and Workshop refers to the study and practical implementation of programming techniques, hardware interfacing, and control algorithms used to design and operate robotic systems. It combines embedded systems, sensors, actuators, and software to enable robots to perform specific tasks efficiently. In this project, robotics programming concepts are applied to interpret hand gestures and control the movement of a robotic arm using microcontrollers and single-board computers.

3.2 Features of Robotics Programming

(a) **Intelligence** – Enables robots to process sensor data and respond intelligently to user commands.

Real-Time Control – Allows immediate response to input signals such as hand gestures.

(b) **Sensor Integration** – Supports interfacing of sensors like flex sensors and accelerometers.

(c) **Actuator Control** – Provides precise control of servo motors for robotic arm movement.

(d) **Flexibility** – Robotic systems can be reprogrammed for different tasks and applications.

(e) **Scalability** – Supports expansion by adding more sensors or degrees of freedom.

(f) **Reliability** – Ensures stable and accurate operation through efficient programming and calibration.

3.3 Advantages of Robotics Programming

- **Automation** – Enables robots to perform tasks automatically with minimal human effort.
- **Accuracy** – Provides precise control over robotic movements.
- **Repeatability** – Ensures consistent performance for repetitive tasks.
- **Real-Time Operation** – Allows immediate execution of commands.
- **Learning and Innovation** – Enhances hands-on understanding of robotics and embedded systems.

3.4 Disadvantages of Robotics Programming

- (a) **Complexity** – Requires knowledge of both hardware and software.
- (b) **Initial Cost** – Hardware components may increase the initial setup cost.
- (c) **Maintenance** – Robotic systems require regular calibration and maintenance.
- (d) **Skill Requirement** – Needs skilled personnel for programming and troubleshooting.

3.5 Application Areas of Robotics Programming

Robotics programming is widely used in various fields such as:

- Industrial automation and manufacturing
- Medical and assistive robotics
- Educational and research laboratories
- Defense and surveillance systems

Remote handling in hazardous environments

3.6 Technologies and Components Used in Robotics Programming

- (a) **Microcontrollers (ESP32)** – Used for sensor data acquisition and wireless communication.
- (b) **Single Board Computers (Raspberry Pi)** – Used for data processing and control logic.
- (c) **Sensors** – Flex sensors
- (d) **Actuators** – Servo motors for robotic arm movement.
- (e) **Programming Languages** – Python and Embedded C for control and communication.
- (f) **Communication Protocols** – Bluetooth or Wi-Fi for wireless data transmission

Chapter 04: Hardware Requirements

This chapter describes the hardware components used in the development of the **Haptic Feedback using Robotic Arm and Raspberry Pi** along with their features and configurations. Each component plays a vital role in capturing hand gestures, processing data, and controlling the robotic arm movements.

4.1 Raspberry Pi 4

The Raspberry Pi 4 is a powerful single-board computer widely used in robotics and embedded system applications. It supports Linux-based operating systems and provides sufficient processing capability to handle sensor data processing, control algorithms, and communication tasks. In this project, the Raspberry Pi acts as the main controller responsible for processing gesture data and generating control signals for the robotic arm.

4.1.1 Features of Raspberry Pi 4

- Processor: Quad-core ARM Cortex-A72 (64-bit) @ 1.5 GHz
- RAM: 8 GB variants
- Operating Voltage: 5 V (USB-C power supply)
- Connectivity: Bluetooth 5.0
- USB Ports: 2 × USB 3.0 and 2 × USB 2.0
- GPIO Pins: 40-pin header for peripheral interfacing
- Video Output: Dual micro-HDMI ports supporting up to 4K resolution
- Storage: microSD card for operating system and data storage

➤ Raspberry Pi 4 Pinout Correction

In the text provided, Pin 5 was listed as GND, but on a Raspberry Pi 4, **Pin 5 is GPIO3 (SCL)** and **Pin 6 is GND**.

Pin No.	Label	Function
1	3.3V	Power Supply ¹¹¹¹ +2
2	5V	Power Supply (for Servos/Pi) ²²²² +1
3	GPIO 2 (SDA)	I2C Data for PCA9685
5	GPIO 3 (SCL)	I2C Clock for PCA9685
6	GND	Ground
7	GPIO 18	Vibration Motor Control

➤ **Vibration Motor:**

A coin-type vibration motor is used as the primary feedback device. The vibration motor is mounted on the wearable glove and provides tactile feedback to the user based on the force detected by the load cell. Increased gripping force results in stronger or longer vibration, allowing the user to perceive the amount of force applied by the robotic arm.

➤ **Software-Based Display (RealVNC Interface)**

System status, sensor readings, and debugging information are monitored on the Raspberry Pi desktop using RealVNC. This software-based interface acts as a virtual display during development and testing, eliminating the need for an external LCD or monitor.

➤ Flex Sensors & ADC Configuration



FIG 2.1(Flex Sensor)

In the proposed haptic feedback robotic arm system, flex sensors generate analog voltage signals corresponding to finger bending movements. Instead of using an external ADC, the system utilizes the built-in Analog-to-Digital Converter (ADC) of the ESP32 microcontroller to read these signals.

- **Mapping:**
 - **Index:** Flex Sensor 1
 - **Middle:** Flex Sensor 2
 - **Ring:** Flex Sensor 3

➤ Wireless Glove (ESP32) & Power

The **ESP32** is the heart of the wearable glove.

- **Transmission:** It reads the sensors and transmits data via Bluetooth or Wi-Fi to the Raspberry Pi.
- **Charging:** The **TP4056 module** manages the Li-ion battery, protecting it from over-discharging while the user moves the glove.

➤ Servo Motor Control

While the Pi processes the data, the actual movement is handled by the **PCA9685 Driver**⁴⁴⁴.

- **Purpose:** It offloads the PWM generation from the Pi's CPU to prevent jitter in the **4 Servo Motors**.

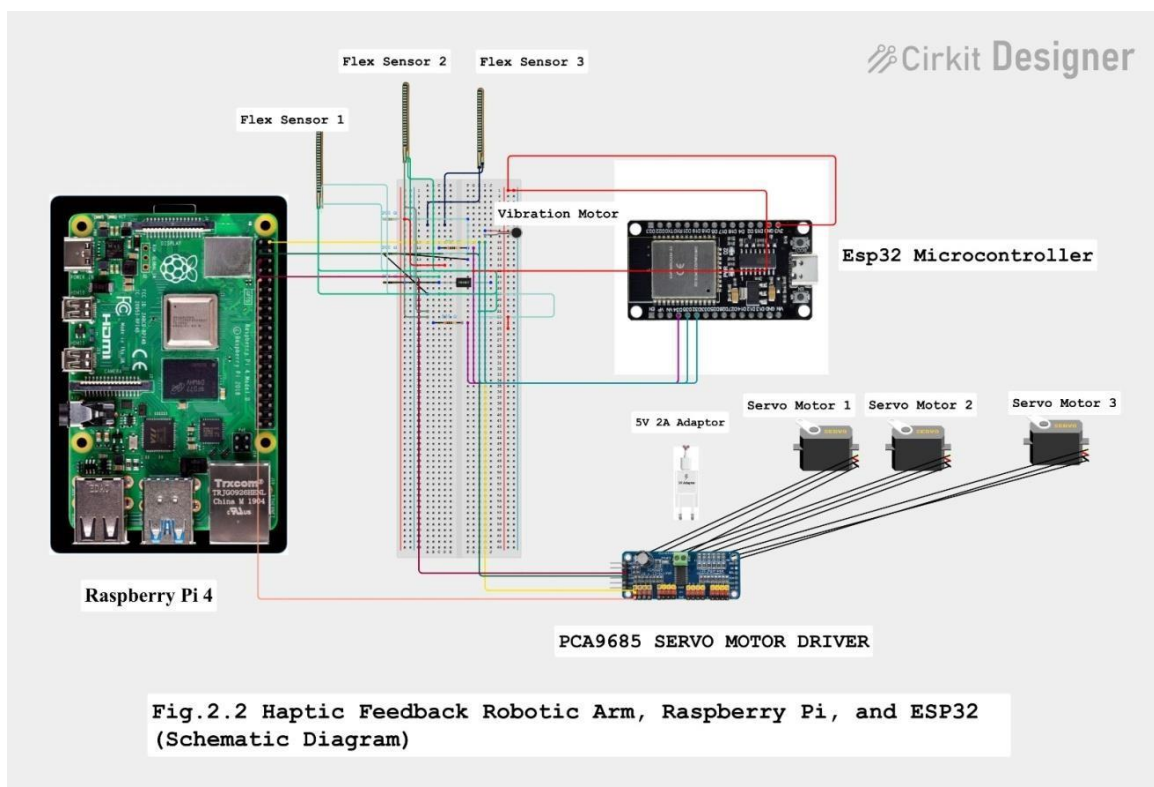


Figure: 2.2 (Schematic Diagram)

4.1.2 Working of the system

1. Flex sensors detect finger bending movements.
2. Esp32 sends the data to Raspberry pi which is connected by Bluetooth
3. The Raspberry pi shows the data
4. The Robotic Arm shows movement according to the finger movement
5. Then a Vibration Motor is insert inside the Ring Finger.
6. After The Robotic Arm Movement the ring Finger Vibrates.

4.1.3 Circuit Diagram

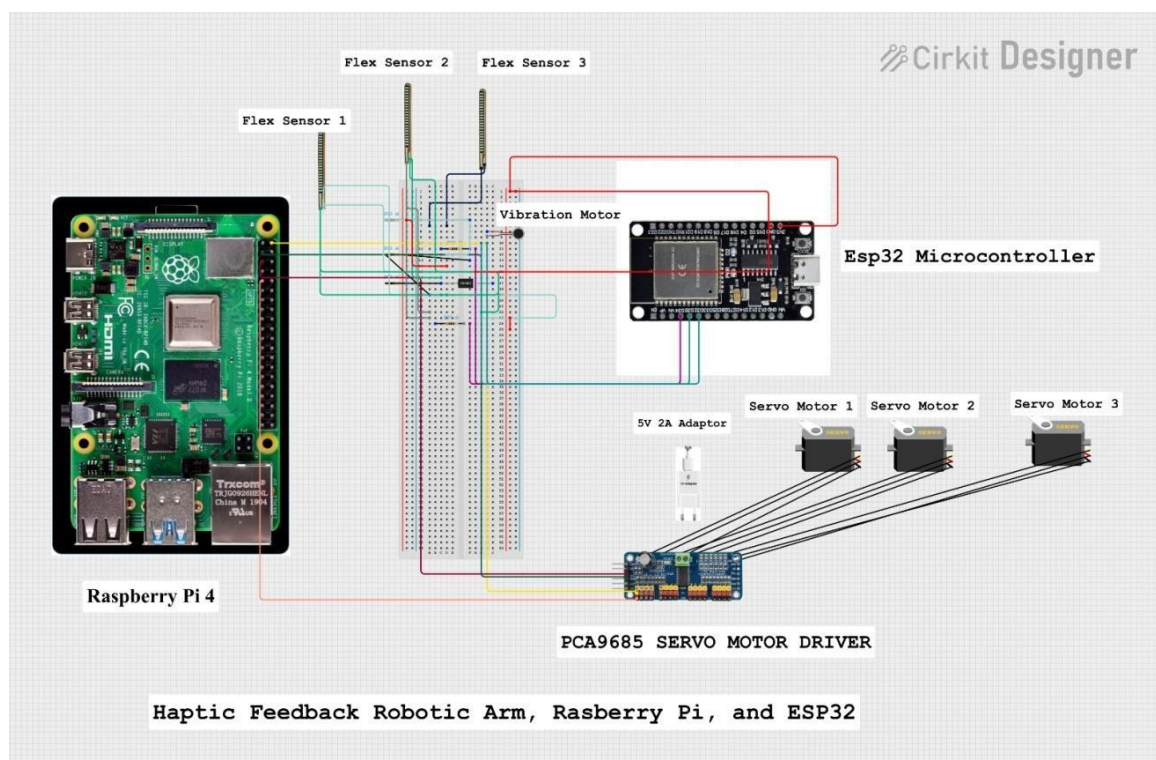


Figure 3.1: Circuit diagram

4.1.4 Components Required

Table 1. Component listing.

Sl. No.	Component and Specification	Quantity
1.	Raspberry Pi 4 (4GB) and USB C 15.3W Adaptor	1
2.	ESP32 Wi-Fi/Bluetooth Module	1
3.	Flex Sensor	3
4.	Accrylic 4-DOF Robotic Arm	1
5.	Mino Servo SG90	4
6.	PCA9685 Servo Driver	1
7.	Vibration Motor, Transistor, Diode	1
8.	microSD Card (32GB)	1
9.	Breadboard / PCB	1
10.	Jumper Wires	80
11.	Power Supply 5V 3A (USB-C)	1

Chapter 05: Software Requirements

5.1 Thonny IDE (Python)

Thonny IDE is a lightweight and beginner-friendly Integrated Development Environment (IDE) designed specifically for Python programming. It is the default IDE provided with Raspberry Pi OS and is well-suited for embedded and educational projects.

In this project, Thonny IDE was used to develop and test Python scripts on the Raspberry Pi for **Bluetooth data reception from the ESP32, dataset creation using one flex sensor values, data preprocessing**

The IDE provides a simple and clean interface for writing, running, and debugging Python programs, making it easier to monitor incoming Bluetooth data and troubleshoot runtime issues during development.

5.1.1 Advantages of Using Thonny IDE

- Pre-installed on Raspberry Pi OS, eliminating the need for additional installation.
- Simple and intuitive user interface suitable for beginners.
- Integrated Python interpreter and debugging tools.
- Suitable for real-time testing and monitoring of data received from two flex sensors.
- Supports popular Python libraries such as **NumPy, Pandas, Scikit-learn, and PySerial**.
- Thonny IDE simplified the development process and enabled efficient debugging and testing of the system software.

5.2 Logic and Flowchart

The software logic of the proposed system follows a structured sequence to ensure accurate real-time hand gesture recognition. The process begins with **data acquisition from two flex sensors connected to the ESP32** and ends with **Haptic Feedback on the Raspberry Pi**.

Logic Flow:

1. Flex sensors detect finger bending movements.
2. Esp32 sends the data to Raspberry pi which is connected by Bluetooth
3. The Raspberry pi shows the data
4. The Robotic Arm shows movement according to the finger movement
5. Then a Vibration Motor is insert inside the Ring Finger.
6. After The Robotic Arm Movement the ring Finger Vibrates.

5.2.1 Software Logic

1. Flex sensors detect finger bending movements.
2. Esp32 sends the data to Raspberry pi which is connected by Bluetooth
3. The Raspberry pi shows the data
4. The Robotic Arm shows movement according to the finger movement
5. Then a Vibration Motor is insert inside the Ring Finger.
6. After The Robotic Arm Movement the ring Finger Vibrates.
7. The process repeats continuously for real-time operation.

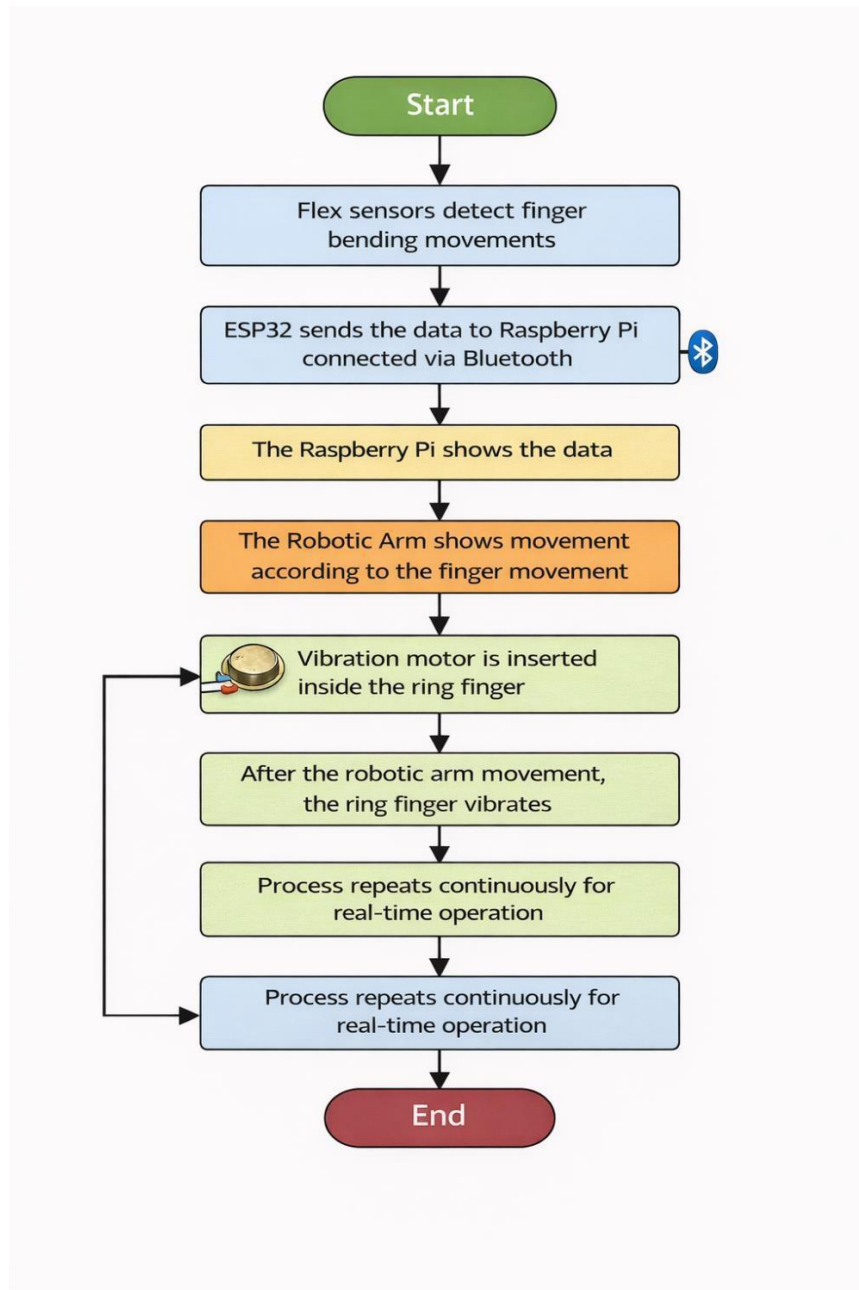


Figure 4.1: Flow chart diagram

Chapter 06: Project development & Testing Aspects

6.1 Project Development Methodology

The development of the haptic feedback-based robotic arm system using Raspberry Pi followed a systematic and modular approach to ensure reliable operation, accurate control, and effective human-robot interaction. The project was divided into multiple stages, including hardware assembly, sensor integration, software development, wireless communication setup, haptic feedback implementation, and system testing.

A modular and iterative development methodology was adopted, where each subsystem—such as the wearable glove unit, force sensing module, robotic arm actuation unit, and control software—was individually developed, tested, and refined before integration into the complete system. This approach enabled early identification of hardware or software issues and ensured smooth coordination between the ESP32, Raspberry Pi, sensors, and actuators.

The iterative process improved system stability, simplified debugging, and resulted in a fully integrated closed-loop haptic feedback robotic arm system capable of real-time operation.

6.2 Hardware Development

The hardware development phase focused on the design, assembly, and integration of the wearable glove unit and the robotic arm hardware required for implementing the haptic feedback system. This phase involved interfacing sensors, microcontroller units, actuators, and power modules to ensure reliable data acquisition, force sensing, and real-time robotic control. Emphasis was placed on creating a compact, stable, and wearable design that supports natural hand movement and effective haptic feedback..

6.2.1 Smart Glove Assembly

- The smart glove was designed to be lightweight, comfortable, and wearable, allowing the user to perform natural hand movements without restriction. Flex sensors were mounted on selected fingers of the glove to accurately measure finger bending movements

sensors capture the user's finger positions and serve as the primary input for controlling the robotic arm.

- The flex sensors were connected to the ESP32 microcontroller, which reads the analog sensor values using its built-in ADC. A voltage divider circuit was used to convert resistance changes of the flex sensors into measurable voltage levels. The ESP32 processes these values and transmits the finger movement data wirelessly to the Raspberry Pi.
- To enable haptic feedback, a coin-type vibration motor was integrated into the glove. The vibration motor activates based on force data received from the robotic arm's gripper, allowing the user to feel tactile feedback corresponding to the applied force. This design completes the closed-loop haptic interaction between the user and the robotic arm.

6.2.2 ESP32 Integration

- The ESP32 microcontroller was used as the main processing and communication unit for the wearable smart glove. It interfaces directly with the flex sensors mounted on the glove and continuously reads their analog values using its built-in Analog-to-Digital Converter (ADC). These analog readings represent the degree of finger bending and are used as control inputs for the robotic arm.
- The ESP32 processes the sensor data by filtering and formatting it into a suitable data structure. This processed gesture data is then transmitted wirelessly to the Raspberry Pi using Bluetooth communication. Wireless transmission eliminates the need for physical connections between the glove and the robotic arm, improving user mobility and system flexibility.

6.2.3 Software Development

The software development process was divided between the ESP32 firmware and the Raspberry Pi software.

6.3.1 ESP32 Firmware Development

The firmware running on the ESP32 was developed using the Arduino framework. It initializes the sensors, reads data at fixed time intervals, and sends the processed sensor values to the Raspberry Pi in a structured format. The firmware also manages Bluetooth connectivity to ensure reliable and continuous data transmission.

6.3.2 Raspberry Pi Software Development

The Raspberry Pi software was developed using Python. It includes programs for receiving Bluetooth data, storing sensor readings in CSV format, preprocessing the dataset, training the machine learning model, and performing real-time hand gesture recognition.

The recognized gestures are **displayed or logged on the Raspberry Pi interface**, eliminating the need for an external display module.

6.4 Dataset Creation and Preprocessing

In the proposed haptic feedback robotic arm system, the focus is on real-time gesture control and force feedback rather than machine learning-based gesture classification. Therefore, a traditional dataset for model training is not required. Instead, sensor data calibration and preprocessing were performed to ensure accurate and stable system operation.

6.4.1 Flex sensor data was recorded during multiple hand movements to determine the minimum and maximum bending values for each finger. These values were used to calibrate the system and establish suitable threshold ranges for controlling the servo motors of the 4- DOF acrylic robotic arm.

6.4.2 The raw sensor readings were processed to reduce noise and fluctuations by applying simple filtering and averaging techniques. Invalid or unstable readings caused by sudden hand movement or loose connections were ignored

6.5 Control Logic and Real-Time Mapping

6.5.1 In the proposed haptic feedback robotic arm system, machine learning techniques are not used. Instead, the system operates based on real-time control logic and direct sensor-to-actuator mapping, which ensures fast response and low computational complexity.

6.5.2 The finger movement data obtained from the flex sensors is read by the ESP32 microcontroller and transmitted wirelessly to the Raspberry Pi. The Raspberry Pi processes the incoming sensor values and maps them to corresponding servo motor angles using predefined threshold ranges and scaling functions.

6.6 System Integration

6.6.1 After individual testing of the hardware and software modules, the complete haptic feedback robotic arm system was integrated. The ESP32 microcontroller and Raspberry Pi were connected using Bluetooth communication, and real-time transmission of flex sensor data from the wearable glove to the Raspberry Pi was successfully verified.

6.6.2 The Raspberry Pi received the live sensor data and processed it using real-time control logic to generate PWM signals for controlling the servo motors of the 4-DOF acrylic robotic arm through the PCA9685 driver. Simultaneously, force data from the load cell mounted on the gripper was processed to generate haptic feedback through the vibration motor on the glove.

6.7 Testing and Results

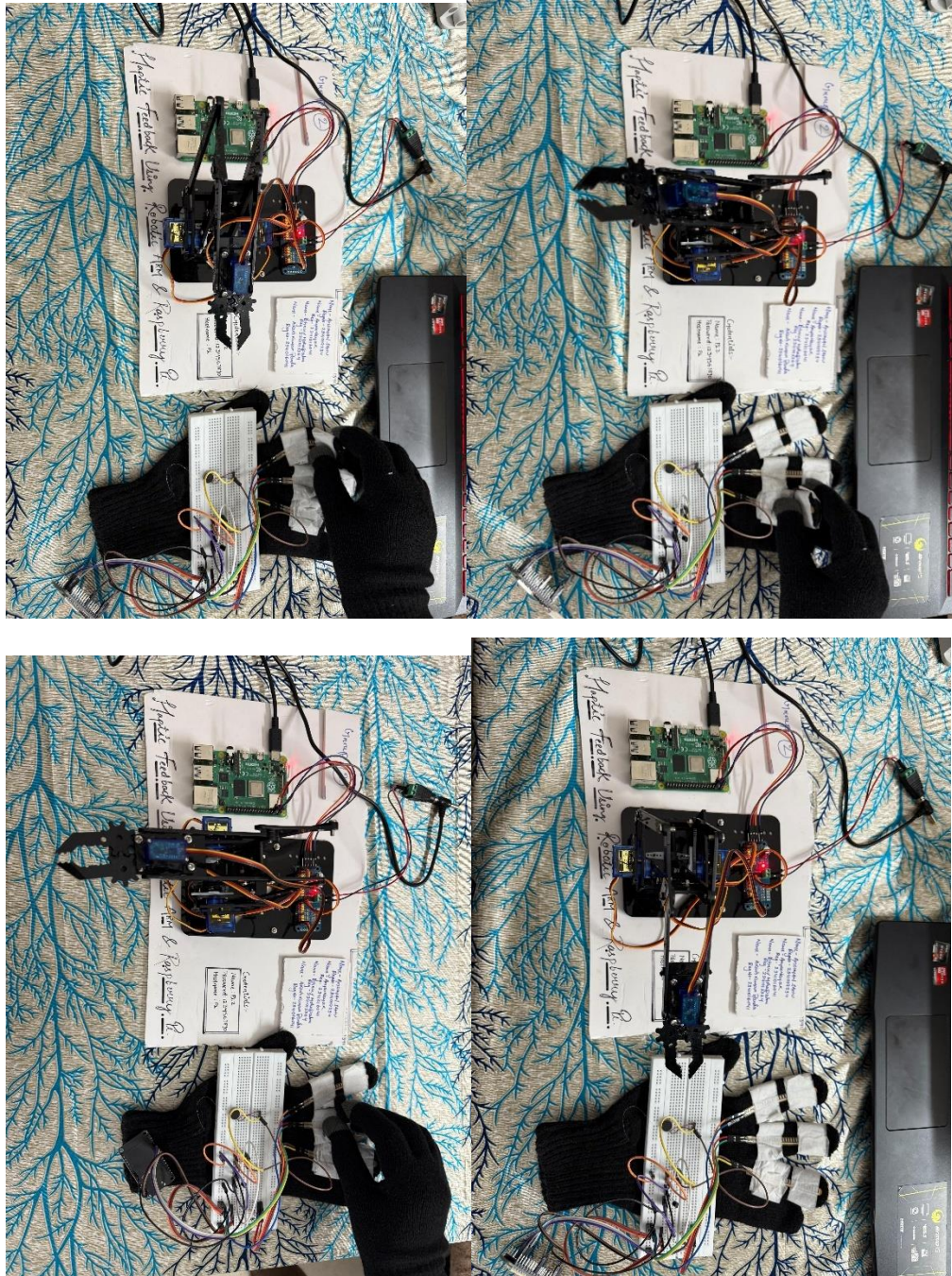


Figure 6.7: Testing and Results of our Prototype

6.7.1 Unit Testing

Each module was tested independently:

Each module of the haptic feedback robotic arm system was tested individually. Flex sensors were verified for accurate finger movement detection using the ESP32 ADC, Bluetooth communication was tested for reliable data transfer to the Raspberry Pi, the PCA9685 servo driver and servo motors were checked for smooth and precise arm movement, the vibration motor was tested for proper haptic feedback, and the buck converter was validated for stable voltage regulation.

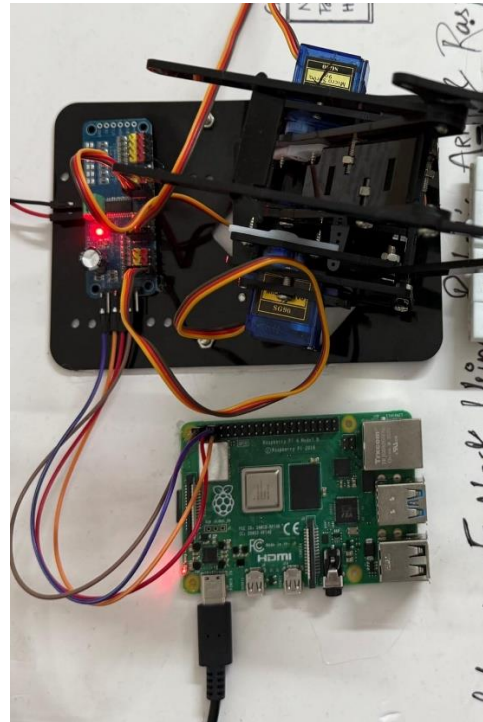
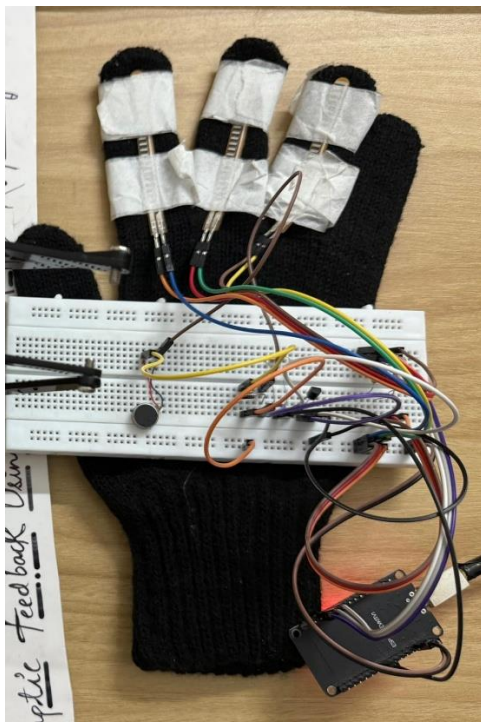


Figure 6.7.1: Individually Testing of Glove and Robotic Arm

6.7.2 Integration Testing

- After successful unit testing, all hardware and software modules were integrated. The ESP32, Raspberry Pi, flex sensors, PCA9685 servo driver, servo motors, vibration motor, and buck converter were tested together to verify seamless Bluetooth communication, synchronized robotic arm movement, effective haptic feedback, and stable power operation under real-time conditions.

6.7.3 Performance Testing

- The integrated haptic feedback robotic arm system was tested to evaluate real-time responsiveness, accuracy of finger movement detection, smoothness of servo motor operation through the PCA9685 driver, reliability of Bluetooth communication, effectiveness of haptic feedback, and stability of power supply using the buck converter under continuous operation.

6.7.4 Result Analysis

- The experimental results demonstrate that the haptic feedback robotic arm successfully replicates user finger movements in real time using flex sensor inputs. Bluetooth communication between the ESP32 and Raspberry Pi remained stable with minimal delay. The PCA9685 servo driver provided smooth and precise control of the 4-DOF acrylic robotic arm. The vibration motor delivered effective tactile feedback, enhancing user awareness during operation, while the buck converter ensured stable voltage supply. Overall, the system operated reliably and met the intended design objectives

6.8 Summary

- The haptic feedback robotic arm system using Raspberry Pi and ESP32 was successfully designed and implemented. Flex sensors enabled intuitive gesture control, while the PCA9685 servo driver ensured smooth robotic arm movement. Stable Bluetooth communication, effective vibration-based haptic feedback, and reliable power regulation using a buck converter resulted in a responsive and user-friendly system suitable for educational and assistive robotic applications.

Chapter 07: Conclusion & Future Scope

7.1. Result

The implementation of the haptic feedback robotic arm system using Raspberry Pi and ESP32 was successfully completed, and the system performed as intended during testing and evaluation. Finger movement data captured through flex sensors mounted on a wearable glove was accurately read by the ESP32 and transmitted wirelessly to the Raspberry Pi with minimal latency. This enabled real-time interpretation of user gestures and smooth replication of hand movements by the 4-DOF acrylic robotic arm.

The use of the PCA9685 PWM servo driver ensured precise and jitter-free control of multiple servo motors, resulting in coordinated and stable robotic arm motion. Each joint of the robotic arm responded proportionally to finger movements, demonstrating effective mapping between sensor inputs and actuator outputs. The Bluetooth communication link remained stable throughout continuous operation, allowing uninterrupted control without data loss.

Haptic feedback was successfully provided using a coin-type vibration motor integrated into the glove. The vibration response enhanced user awareness during robotic arm operation and contributed to improved control confidence. Additionally, the buck converter provided regulated voltage to the system components, ensuring electrical stability and preventing performance degradation during extended use.

Overall, the experimental results confirm that the proposed system achieves responsive, reliable, and intuitive human–robot interaction. The project effectively demonstrates the practical integration of embedded systems, wireless communication, and haptic feedback in a low-cost robotic application, meeting all the intended objectives.

7.2. Limitations

Although the proposed haptic feedback robotic arm system performs effectively, it has certain limitations that can be addressed in future work. The system relies on flex sensors for gesture input, which may show variation in readings over time due to sensor wear, temperature changes, or differences in hand size among users. This can affect long-term accuracy and may require periodic recalibration.

The haptic feedback mechanism is implemented using a vibration motor, which provides tactile feedback but does not offer precise force magnitude representation. As a result, the feedback is limited to vibration intensity rather than true force-based resistance.

The system uses Bluetooth communication, which has a limited operating range and may experience interference in environments with multiple wireless devices. Additionally, the 4-DOF acrylic robotic arm has mechanical constraints, limiting its load capacity and range of motion compared to industrial robotic arms.

Finally, the absence of advanced control algorithms or adaptive learning techniques restricts the system's ability to automatically adjust to different users or complex manipulation tasks.

7.3 Future Enhancement and Scope

- The proposed haptic feedback robotic arm system provides a strong foundation for further improvements and advanced applications. Several enhancements can be incorporated in future versions to increase system performance, accuracy, and usability.
- More advanced haptic feedback mechanisms, such as force-sensitive resistors or resistive actuators, can be integrated to provide realistic force-based feedback instead of simple vibration. This would allow users to perceive the exact amount of force applied by the robotic arm during object manipulation.
- The system can be enhanced by increasing the degrees of freedom of the robotic arm to achieve more complex and human-like movements. Using higher-torque servo motors and improved mechanical structures would also enable the arm to handle heavier objects.
- Wireless communication can be upgraded from Bluetooth to Wi-Fi or IoT-based platforms, enabling remote monitoring and control of the

robotic arm over the internet. This would make the system suitable for teleoperation and remote assistance applications.

- Advanced control algorithms or adaptive learning techniques can be implemented to improve motion smoothness and automatically adjust to different users. Integration with computer vision or sensor fusion techniques could further enhance interaction accuracy

References

- [1] J. J. Craig, *Introduction to Robotics: Mechanics and Control*, 3rd ed., Pearson Education, 2005.
- [2] S. Haddadin, A. De Luca, and A. Albu-Schäffer, “Robot collisions: A survey on detection, isolation, and identification,” *IEEE Transactions on Robotics*, vol. 33, no. 6, pp. 1292–1312, 2017.
- [3] R. S. Dahiya, G. Metta, M. Valle, and G. Sandini, “Tactile sensing— from humans to humanoids,” *IEEE Transactions on Robotics*, vol. 26, no. 1, pp. 1–20, 2010.
- [4] Raspberry Pi Foundation, “Raspberry Pi 4 Model B Documentation,” [Online]. Available: <https://www.raspberrypi.org/documentation/>
- [5] Espressif Systems, “ESP32 Series Datasheet,” [Online]. Available: <https://www.espressif.com/en/products/socs/esp32>
- [6] NXP Semiconductors, “PCA9685 16-Channel, 12-Bit PWM LED Controller Datasheet,” [Online].
- [7] Arduino, “Arduino IDE Documentation,” [Online]. Available: <https://www.arduino.cc/en/software>
- [8] M. Cutkosky and W. Provancher, “Haptic exploration and manipulation,” *Springer Handbook of Robotics*, Springer, 2016.
- [9] J. K. Salisbury and M. A. Srinivasan, “Phantom-based haptic interaction with virtual objects,” *IEEE Computer Graphics and Applications*, vol. 17, no. 5, pp. 6–10, 1997.

Appendix 01

ESP32 AND RASPBERRY PIBLUETOOTH CONNECTION CODE:

```
import bluetooth

# Replace with ESP32 Bluetooth name
target_name = "ESP32_Flex"

print("Searching for ESP32 device...")

nearby_devices = bluetooth.discover_devices(duration=8,
lookup_names=True)

esp32_address = "00:70:07:84:15:2E"
for addr, name in nearby_devices:
    if target_name == name:
        esp32_address = addr
        break

if esp32_address is None:
```

```
print("Could not find ESP32_Flex device")
exit(1)

print(f"Found ESP32 at {esp32_address}, connecting...")

# Connect to ESP32 Bluetooth SPP
sock = bluetooth.BluetoothSocket(bluetooth.RFCOMM)
sock.connect((esp32_address, 1)) # Channel 1 usually works

print("Connected. Receiving data...\n")

try:
while True:
data = sock.recv(1024).decode("utf-8").strip()
if data:
# Data format: "flex1,flex2,flex3"
values = data.split(",")
if len(values) == 3:
print(f"Index: {values[0]} | Middle: {values[1]} | Ring: {values[2]}")
else:
print("Raw:", data)
except KeyboardInterrupt:
print("\nDisconnected.")
sock.close()
```

SYSTEM ID SERVICES

.rfcomm service

/etc/systemd/system/rfcomm-glove.service

[Unit]

Description=RFCOMM Bind for ESP32 Glove

After=bluetooth.service

[Service]

Type=oneshot

RemainAfterExit=yes

ExecStart=/usr/bin/rfcomm bind 0 ESP32_GLOVE

ExecStop=/usr/bin/rfcomm release 0

[Install]

WantedBy=multi-user.target

PERMISSIONS

sudo usermod -aG bluetooth pi

sudo usermod -aG dialout pi

sudo reboot

Full Working Code For Haptic Feedback Robotic Arm

```
import bluetooth
import RPi.GPIO as GPIO
import time

# ----- CONFIG -----
ESP32_NAME = "ESP32_Flex"
RING_THRESHOLD = 3000
VIBRATION_GPIO = 18
# -----

# GPIO Setup
GPIO.setmode(GPIO.BCM)
GPIO.setup(VIBRATION_GPIO, GPIO.OUT)
GPIO.output(VIBRATION_GPIO, GPIO.LOW)

# ----- BLUETOOTH -----
print("Searching for ESP32...")
devices = bluetooth.discover_devices(duration=8, lookup_names=True)

esp32_addr = None
for addr, name in devices:
    if name == ESP32_NAME:
```

```
    esp32_addr = addr
    break

if esp32_addr is None:
    print("ESP32 not found")
    GPIO.cleanup()
    exit(1)

sock = bluetooth.BluetoothSocket(bluetooth.RFCOMM)
sock.connect((esp32_addr, 1))
print("Connected to ESP32")

# ----- MAIN LOOP -----
try:
    while True:
        data = sock.recv(1024).decode().strip()
        if not data:
            continue

        values = data.split(",")
        if len(values) != 3:
            continue

        ring = int(values[2])
        print("Ring:", ring)
```

```
    if ring <= RING_THRESHOLD:
        GPIO.output(VIBRATION_GPIO, GPIO.HIGH)
        print("🔔 VIBRATION ON")
    else:
        GPIO.output(VIBRATION_GPIO, GPIO.LOW)
        print("🔕 VIBRATION OFF")

except KeyboardInterrupt:
    print("\nStopping...")
finally:
    GPIO.output(VIBRATION_GPIO, GPIO.LOW)
    GPIO.cleanup()
    sock.close()
```

Appendix 02

A02.1. Project Proposal Form

- The project proposal form was prepared and duly signed from our Faculty-in-Charge **Dr. Shaktijeet Mohapatra and Dr. Bhanja Kishor Swain**. The same is attached at the last of this report.

A02.2. Project Management

#	Component	Individual Contributions in %			Total
		Name 01	Name 02	Name 03	
1.	Planning	Dhruv Mohapatra (35%)	Akash Kumar Panda (35%)	Ansuman Sahu (30%)	100%
2.	Background Research and Analysis	Priyanshu Priyadarshan Sahu (40%)	Pratyush Kumar Mohanty (30%)	Aryan Nayak (30%)	100%
3.	Hardware design	Akash Kumar Panda (50%)	Dhruv Mohapatra (20%)	Priyanshu Priyadarshan Sahu (30%)	100%
4.	Software design	Dhruv Mohapatra (50%)	Akash Kumar Panda (30%)	Aryan Nayak (20%)	100%
5.	Data Collection	Ansuaman Sahu (45%)	Aryan Nayak (30%)	Dhruv Mohapatra (25%)	100%
6.	Testing	Dhruv Mohapatra (35%)	Akash Kumar Panda (35%)	Priyanshu Priyadarshan Sahu (30%)	100%
7.	Final Assembling	Akash Kumar Panda (40%)	Dhruv Mohapatra (30%)	Aryan Nayak (30%)	100%
8.	Project report writing	Dhruv Mohapatra (60%)	Aryan Nayak (20%)	Anssuman Sahu (20%)	100%
9.	Presentation	Akash Kumar Panda (40%)	Pratyush Kumar Mohanty (40%)	Dhruv Mohapatra (20%)	100%

10.	Logistics	Aryan Nayak (30%)	Akash Kumar Panda (60%)	Pratyush Kumar Mohanty (10%)	100%
11.	Demonstration Video	Akash Kumar Panda (35%)	Priyanshu Priyadarshan Sahu (45%)	Dhruv Mohapatra (20%)	100%

A02.3. Bill of Material

Table 1. Component listing.

Sl. No.	Component Name	Specification / Description	Quantity	Unit Cost (Rs.)	Total Cost (Rs.)
1	Raspberry Pi 4 Model B	4 GB RAM	1	6,649	6,649
2	Power Supply Adapter	Official USB-C 15.3 W (5 V) for Raspberry Pi	1	769	769
3	Micro SD Card	32 GB SanDisk Ultra, Class 10	1	499	499
4	Robotic Arm Kit	DIY 4-DOF Acrylic Robotic Arm (without servos)	1	469	469
5	Breadboard	GL12 – 840 Points Solderless Breadboard	1	60	60
6	Flex Sensors	2.2" Bend Sensor	3	299	897
7	Servo Motors	SG90 Mini Servo, 180° Rotation	3	99	297
8	PWM Servo Driver	PCA9685, 16-Channel, 12-bit, I2C	1	349	349
9	PWM Servo Driver (Extra)	PCA9685, 16-Channel, 12-bit	2	350	700
10	Microcontroller	ESP32 30-Pin CP2102 (Wi-Fi + Bluetooth)	1	349	349
11	Jumper Wires (M–F)	20 cm Male–Female (Set of 40)	1	49	49
12	Jumper Wires (M–M)	20 cm Male–Male (Set of 40)	1	50	50

Sl. No.	Component Name	Specification / Description	Quantity	Unit Cost (Rs.)	Total Cost (Rs.)
13	Vibration Motor	Coin Type Mobile Phone Vibrator Motor	1	39	39
14	Transistor	2N2222A NPN	1	5	5
15	Diode	1N4007	2	2	4
16	Rechargeable Battery	18650 Li-Ion Cell, 3.7 V, 2200 mAh	1	79	79
17	Battery Holder	1 × 18650 Holder	1	18	18
18	Buck / Power Module	5 V Micro USB Step-Up Power Bank Module	1	39	39
19	Power Adapter	5 V, 2 A DC Adapter	1	250	250
20	DC Female Connector	Female DC Jack Converter for Adapter	1	40	40

Total Cost : 11,611 Rupees

Appendix 03

A03.1 Data Sheets

This appendix provides the technical specifications and reference data sheets of the major hardware components used in the development of the Haptic Feedback Robotic Arm System using Raspberry Pi and ESP32. These data sheets are essential for understanding the electrical characteristics, operating conditions, pin configurations, communication interfaces, and functional capabilities of each component used in the system.

The data sheets included in this appendix cover key components such as the Raspberry Pi 4 Model B, ESP32 microcontroller, PCA9685 PWM servo driver, SG90 servo motors, flex sensors, vibration motor, and power regulation modules (buck converter and adapters). They serve as a reference during system design, hardware interfacing, power management, and troubleshooting.

- **A03.1.1 ESP32 Microcontroller**

The ESP32 is a low-power system-on-chip (SoC) microcontroller with integrated Wi-Fi and Bluetooth capabilities. It serves as the core processing and communication unit of the smart glove.

Key Specifications:

- Microcontroller: Tensilica Xtensa LX6 (Dual-core)
- Operating Voltage: 3.3 V
- Clock Frequency: Up to 240 MHz
- GPIO Pins: Up to 34 programmable pins
- Communication Interfaces: UART, SPI, I2C, PWM
- Wireless Connectivity: Wi-Fi 802.11 b/g/n and Bluetooth v4.2
- Power Consumption: Supports low-power sleep modes

Application in Project:

Used to read sensor data from flex sensors and MPU6050, process the data, and transmit it wirelessly to the Raspberry Pi via Bluetooth.

- **A03.1.2 Flex Sensors**

Flex sensors are resistive sensors whose resistance varies with bending. They are used to detect finger movement in the smart glove.

Key Specifications:

- Operating Voltage: 3.3 V – 5 V
- Resistance (Flat): $\sim 10\text{ k}\Omega$
- Resistance (Bent): $\sim 30\text{--}40\text{ k}\Omega$
- Response Time: $< 1\text{ ms}$
- Length: 2.2 inches / 4.5 inches

Application in Project:

Two flex sensors are mounted on selected fingers to detect finger bending and gesture variations.

- **A03.1.3 Raspberry Pi 4 Model B**

The Raspberry Pi 4 Model B acts as the main processing unit for data analysis, machine learning inference, and system monitoring.

Key Specifications:

- Processor: Quad-core ARM Cortex-A72
- RAM: 4 GB (varies by model)
- Operating Voltage: 5 V
- Operating System: Raspberry Pi OS
- Connectivity: Bluetooth, Wi-Fi, USB, HDMI, GPIO

Application in Project:

Used for receiving sensor data via Bluetooth, preprocessing data, running the machine learning model, and displaying or logging recognized gestures.

- **A03.1.4 Power Supply (Battery Module)**

A portable power source is used to operate the smart glove system.

Key Specifications:

- Battery Type: Li-ion battery
- Output Voltage: Regulated to 3.3 V
- Capacity: Depends on battery rating

Application in Project:

Provides power to the ESP32 microcontroller and connected sensor modules.

- **A03.1.5 Bluetooth Communication Module (Integrated in ESP32)**

Bluetooth communication is enabled using the ESP32's built-in Bluetooth module.

Key Specifications:

- Bluetooth Version: v4.2 (Classic + BLE)
- Communication Range: Approximately 10 meters
- Data Transmission Rate: Moderate and suitable for real-time sensor data

Application in Project:

Used for wireless transmission of sensor data from the smart glove to the Raspberry Pi.

- **Summary**

This project successfully designed and implemented a Haptic feedback–based robotic arm system using a Raspberry Pi, ESP32, and a 4-DOF acrylic robotic arm. Finger movements captured through flex sensors mounted on a wearable glove were transmitted wirelessly to the Raspberry Pi, enabling real-time and intuitive control of the robotic arm. The PCA9685 PWM servo driver ensured smooth and precise servo motor operation, while the vibration motor provided effective tactile feedback to enhance user interaction. Stable power regulation using a buck converter ensured reliable system performance. Overall, the project demonstrates an efficient, low-cost, and scalable solution for human–robot interaction with haptic feedback, suitable for educational and assistive robotics applications